

PCAD
Programmazione Concorrente
Algoritmi Distribuiti

Arnaud Sangnier
arnaud.sangnier@unige.it

Programmazione Rete in Java

Due modi principali di comunicare

- Comunicazione **per flussi** (streaming) : come TCP
 - Informazioni sono trasmesse in ordine
 - Non c'è perdita dei messaggi
 - Inconvenienti:
 - Necessità una connessione
 - Necessità di risorse per gestire la correttezza della comunicazione
- Comunicazione **per pacchetti** (datagram): come UDP
 - L'ordine nella trasmissione dei pacchetti non è garantito
 - Un pacchetto mandato per prima può arrivare dopo un pacchetto mandato in un secondo tempo
 - Non ci sono garanzie
 - Un pacchetto mandato può essere perso

Paradigma di funzionamento

- Comunicazione **per flussi** (streaming) : come TCP
 - Modello Client-Server
 - Un server attende una connessione
 - Poi comunica con il processo che ha richiesto la connessione

**Avere un server multi-threaded permette di
gestire più connessioni in parallelo**

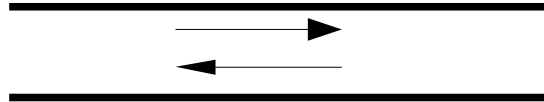
- Comunicazione **per pacchetti** (datagram): come UDP
 - Un processo attende messaggi su un port
 - I messaggi possono arrivare da diversi processi
 - Secondo i messaggi ricevuti fa azioni

Comment vedere TCP

TCP -> comunicazione con i flussi, i.e. comunicazione connessa



Client

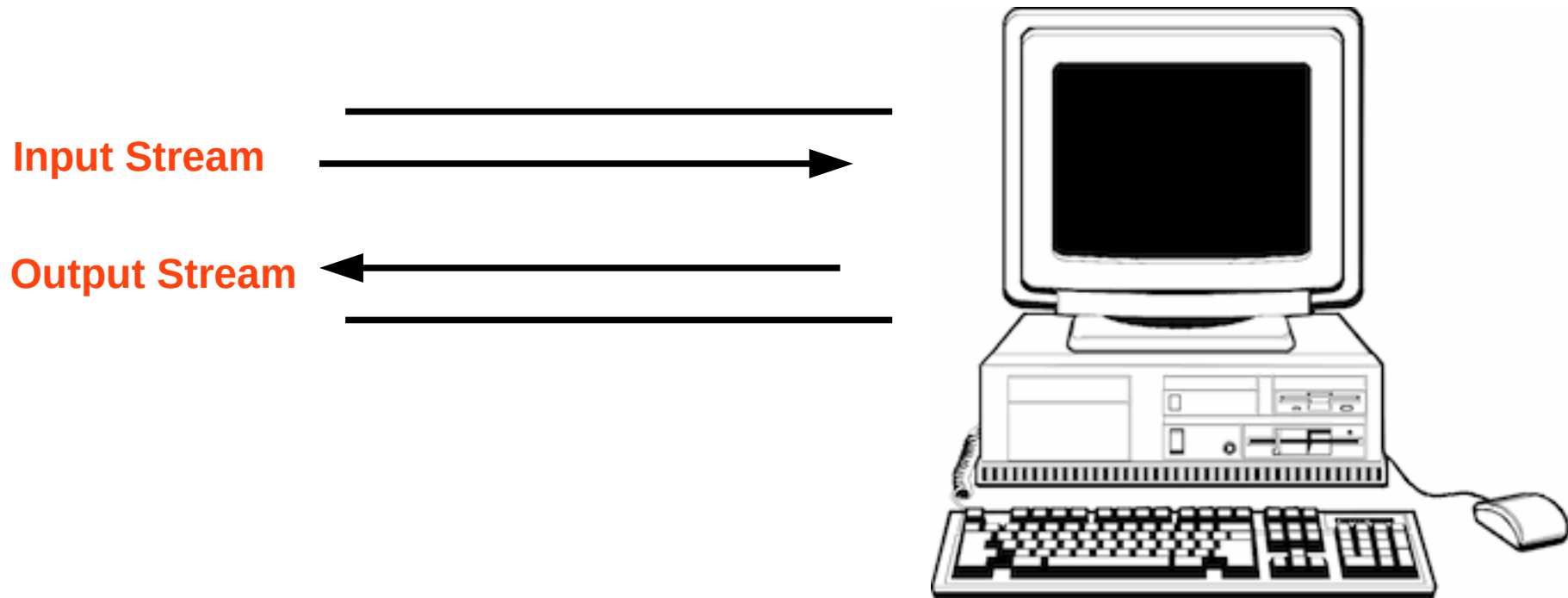


Server

Per creare un sistema TCP

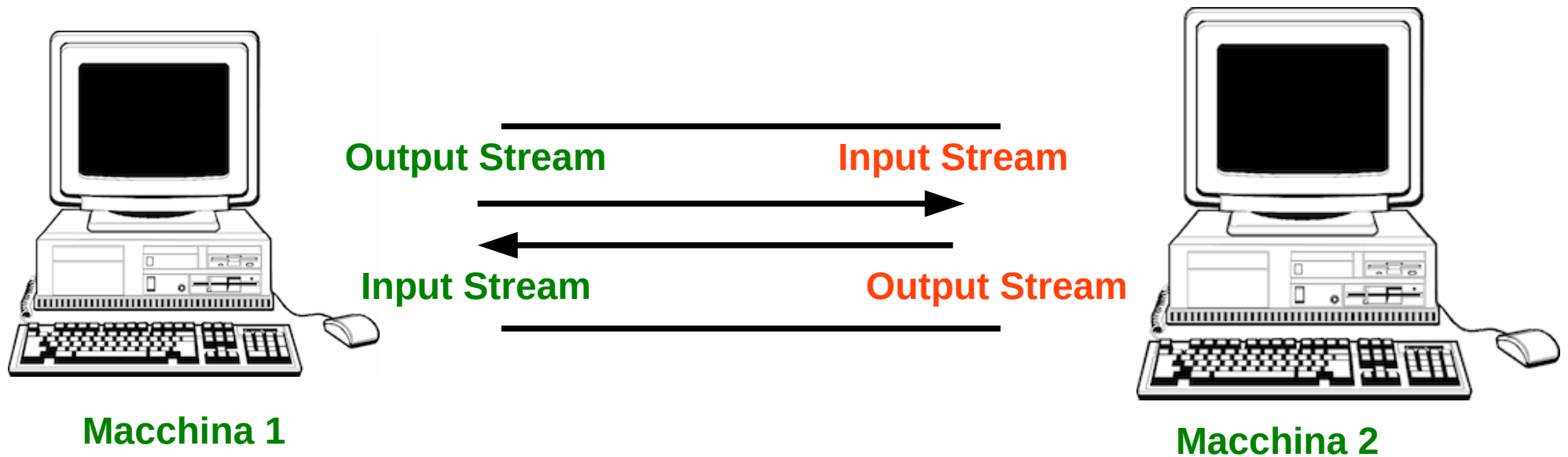
- Come comunicare in TCP in Java
- Come connettersi ad un server (presente su un computer e ascoltando su un port)
 - lato **client**
- Come ricevere dei messaggi
- Come mandare dei messaggi
- Come creare un servizio che ascolta su un port d
 - lato **server**

I flussi nella rete



Dal punto di vista della macchina, il flusso d'Input sono i dati che arrivano e il flusso d'Output, i dati che escono

I flussi nella rete



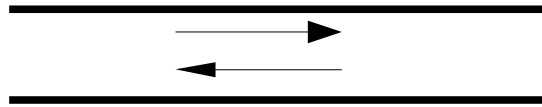
*I dati che escono sul flusso di output della macchina 1
arrivano sul flusso d'input della macchina 2!*

Per il nostro problema

- **TCP -> modalità connessa**



Client



Server

- Come connettersi a un servizio?
- Come inviare dei dati a un servizio?
- Come ricevere i dati inviati?

Legame tra rete e flussi



Perché le funzione sui flussi
ci aiutano ?

- Gli **sockets** permettono di connettersi a macchine distanti e di comunicare tramite flussi
- Quindi è grazie ai flussi che inviamo e riceviamo i dati

Le socket in una slide

- Una socket rappresenta una connessione fra due macchine
- Passa fra due port
- Operazione basiche realizzate con una socket
 - Connettersi ad una macchina distante ad un port specifico
 - Inviare dei dati
 - Ricevere dei dati
 - Chiudere una connessione
 - Connettersi a un port della macchina dove si trova la socket
 - Accettare delle connessioni
- Le ultime due operazioni sono necessarie per il lato server (cf la classe **ServerSocket**)

Esempio semplice

- Programmare un server e un cliente per un servizio echo
- Dopo aver accettato la connessione, Il server attende una string che termina con '\n' e la rimanda al client dopo aver inserito "ECHO" al inizio
- Supponiamo che il server ascolti sul port 4242
- Per l'esempio, il server sarà localhost, cioè il server sarà all'indirizzo IP 127.0.0.1

Cliente per il servizio echo

- Domande da porsi prima di programmare il client:
 - Su quale macchina si trova il servizio con il quale vogliamo comunicare o qual'è il suo indirizzo IP
 - **localhost** o **127.0.0.1**
 - Su quale port è connesso questo servizio
 - **port 4242**
 - Qual è la forma della comunicazione?
 - Il client invia una string terminata con '\n'
 - Il client inizia la conversazione
 - Il server risponde con la stessa string completata con 'ECHO '

Creazione di socket TCP

- Come creare una socket che si connette al port 4242 di localhost
- Uso della classe **java.net.Socket**
 - Ci sono più costruttori possibili
 - Usiamo: :
 - **public Socket(String host, int port) throws UnknownHostException, IOException**
 - **host** è il nome della macchina remota
 - **port** è il numero di port

```
Socket socket=new Socket("localhost", 4242);
```

Creazione di socket TCP

- Cosa succede quando facciamo :

```
Socket socket=new Socket("localhost", 4242) ;
```

- Una socket viene creata tra la macchina locale e **localhost** (per l'esempio facciamo tutto in local, ma in questo caso 'localhost' va inteso come una macchina remota)
- La socket viennte attaccata localmente a un port temporaneo (scelto dalla JVM)
- Al momento della creazione della socket, viene inviata una richiesta di connessione a **localhost** sul **port 4242**
- Se la connessione viene accettata, il costruttore termina normalmente
- A questo punto, è possibile usare la socket per comunicare

Essere attento alle eccezione

- Il prototipo per creare una socket TCP è:
 - **public Socket(String host, int port) throws UnknownHostException, IOException**
- È importante recuperare le eccezioni sollevate per evitare che un programma fallisca in un modo brutale
- È ancora più importante nella programmazione di rete perché i problemi che possono verificarsi sono **numerosi**:
 - numero di port errato, host sconosciuto, problemi con la rete, con i flussi, etc

```
try{...}  
catch (Exception e) {  
    System.out.println(e);  
    e.printStackTrace();  
}
```

Esempio di connessione

```
import java.net.*;
import java.io.*;
public class ClientEcho{
    public static void main(String[] args){
        try{
            Socket socket=new Socket("localhost",4242);
            socket.close();
        }
        catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
    }
}
```


Uso della socket creata

- Per chiudere la connessione:
 - **void close()**
- Per comunicare tramite la socket:
 - **public InputStream getInputStream() throws IOException**
 - per recuperare il flusso d'input per i dati che riceviamo
 - **public OutputStream getOutputStream() throws IOException**
 - per recuperare il flusso d'output per i dati che mandiamo
- **Osservazione:**
 - La comunicazione sulle sockets è **bi-direzionale**
 - Si usa la stessa socket per ricevere e inviare dei dati
 - **Ma i flussi sono diversi**

Ricuperare i flussi

- Ecco come recuperare i flussi :

```
Socket socket=new Socket("localhost", 4242);  
InputStream is=socket.getInputStream();  
OutputStream os=socket.getOutputStream();
```

- E per usare i filtri che rendono l'uso dei flussi più semplice

```
Socket socket=new Socket("localhost", 4242);  
BufferedReader br=new BufferedReader(  
    new InputStreamReader(socket.getInputStream()));  
PrintWriter pw=new PrintWriter(  
    new OutputStreamWriter(socket.getOutputStream()));
```

Funzionamento del nostro client

- 1) Creare una socket per connettersi
- 2) Recuperare i flussi di input/output
- 3) Inviare la string "Bla1b1a"
- 4) Aspettare di ricevere la string di risposta
- 5) Stampare il messaggio ricevuto
- 6) Chiudere i flussi e la socket

Creazione di un client TCP

```
import java.net.*;
import java.io.*;

public class ClientEcho{
    public static void main(String[] args){
        try{
            Socket socket=new Socket("localhost",4242);
            BufferedReader br=new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter pw=new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            pw.println("Blabla");
            pw.flush();
            String mess=br.readLine();
            System.out.println("Message from the server:"+mess);
            pw.close();
            br.close();
            socket.close();
        }
        catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
    }
}
```

Punti importanti

- Non dimenticare di recuperare le eccezioni
- Utilizzare la funzione `flush()` per svuotare il buffer legato al flusso di output e inviare il messaggio
- La lettura con `readLine()` blocca finché non viene ricevuto un messaggio

E ora ?



E come facciamo il server ?

- Si usa la classe **ServerSocket**
- La cosa pratica
 - a parte l'attesa di connessione
 - la manipolazione dei socket sarà simile a quella dello client

Creazione di un server

- Usiamo la classe **java.net.ServerSocket**
- Esistono anche qua diversi costruttori
- Due funzioni ci interessano
 - **public ServerSocket(int port) throws IOException**
 - Crea un socket che ascolta sul port e serve solo ad aspettare le richieste di connessione

```
ServerSocket server=new ServerSocket(4242);
```

- **public Socket accept() throws IOException**
 - Aspetta una richiesta di connessione
 - Restituisce una socket che servirà alla comunicazione

Et il nome della macchina ?



Perché non diamo il nome
di una macchina al
costruttore ?

```
ServerSocket server=new ServerSocket(4242);
```

- Perché il server si trova sulla macchina in cui viene eseguito il programma
- Il **server** non ha bisogno di conoscere il nome della macchina in cui si trova

Funzionamento del nostro server

- 1) Creare un **ServerSocket** legato al port
- 2) Attendere una richiesta di connessione con **accept**
- 3) Recuperare la socket per la comunicazione
- 4) Recuperare i flussi di input/output
- 5) Comunicare con il client
- 6) Chiudere i flussi e la socket
- 7) Andare al punto 2)

Creazione di un server TCP

```
import java.net.*;
import java.io.*;

public class ServerEcho{
    public static void main(String[] args){
        try{
            ServerSocket server=new ServerSocket(4242);
            while(true){
                Socket socket=server.accept();
                BufferedReader br=new BufferedReader(new InputStreamReader(socket.getInputStream()));
                PrintWriter pw=new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
                String mess=br.readLine();
                System.out.println("Receive message:"+mess);
                pw.println("ECHO "+mess);
                pw.flush();
                br.close();
                pw.close();
                socket.close();
            }
        } catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
    }
}
```

Osservazione

- Il nostro server non accetta più di una connessione alla volta
 - Quando fa un **accept**, comunica con un client specifico e deve avere finito la comunicazione per potere comunicare con un nuovo client
- Come risolvere questo problema?
 - Il serve deve essere **multi-threaded**
 - Deve nello 'stesso' tempo potere:
 - 1) comunicare con un client
 - 2) accettare nuove connessioni
- Il thread principale sarà responsabile per 2)
- Ad ogni nuova connessione si creerà un nuovo thread per comunicare con il nuovo client

Service per il server multi-threaded

```
import java.net.*;
import java.io.*;
import java.lang.*;

public class ServiceEcho implements Runnable{
    public Socket socket;
    public ServiceEcho(Socket s){
        this.socket=s;
    }
    public void run(){
        try{
            BufferedReader br=new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter pw=new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            String mess=br.readLine();
            System.out.println("Receive message:"+mess);
            pw.println("ECHO "+mess);
            pw.flush();
            br.close();
            socket.close();
        }
        catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
    }
}
```

Server multi-threaded

```
import java.net.*;
import java.io.*;

public class ServerEchoConcur{
    public static void main(String[] args){
        try{
            ServerSocket server=new ServerSocket(4242);
            while(true){
                Socket socket=server.accept();
                ServiceEcho serv=new ServiceEcho(socket);
                Thread t=new Thread(serv);
                t.start();
            }
        }
        catch(Exception e){
            System.out.println(e);
            e.printStackTrace();
        }
    }
}
```